Brandon Nguyen, 827813045
COMPE596 - Spring 2026
4/20/2026

COMPE596 P10 - OpenCL Triple Product Vector Addition

1) Code Listings

// host.cpp listing

```
#include "cmdlineparser.h"
#include <iostream>
#include <cstring>
#include <cmath>

#include "xrt/xrt_bo.h"
#include "xrt/xrt_device.h"
#include "xrt/xrt_kernel.h"

#define DATA_SIZE 4096

int main(int argc, char** argv) {
    sda::utils::CmdLineParser parser;

    parser.addSwitch("--xclbin_file", "-x", "input binary file string", "");
    parser.addSwitch("--device_id", "-d", "device index", "0");
    parser.parse(argc, argv);

    std::string binaryFile = parser.value("xclbin_file");
    int device_index = stoi(parser.value("device_id"));

    if (argc < 3) {
        parser.printHelp();
        return EXIT_FAILURE;
    }

    std::cout << "Open the device" << device_index << std::endl;
```

```
auto device = xrt::device(device_index);

std::cout << "Load the xclbin " << binaryFile << std::endl;
auto uuid = device.load_xclbin(binaryFile);


int total_size = DATA_SIZE * 3;
size_t vector_size_bytes = sizeof(float) * total_size;

auto krnl = xrt::kernel(device, uuid, "vadd");

std::cout << "Allocate Buffer in Global Memory\n";
auto boA = xrt::bo(device, vector_size_bytes, krnl.group_id(0));
auto boB = xrt::bo(device, vector_size_bytes, krnl.group_id(1));
auto boC = xrt::bo(device, vector_size_bytes, krnl.group_id(2));
auto boOUT = xrt::bo(device, vector_size_bytes, krnl.group_id(3));

auto A = boA.map<float*>();
auto B = boB.map<float*>();
auto C = boC.map<float*>();
auto OUT = boOUT.map<float*>();

for (int i = 0; i < total_size; i++) {
    A[i] = rand() % 10;
    B[i] = rand() % 10;
    C[i] = rand() % 10;
    OUT[i] = 0;
}

float ref[DATA_SIZE * 3];

for (int i = 0; i < DATA_SIZE; i++) {
    int idx = i * 3;

    float ax = A[idx];
```

```
    float ay = A[idx+1];
    float az = A[idx+2];

    float bx = B[idx];
    float by = B[idx+1];
    float bz = B[idx+2];

    float cx = C[idx];
    float cy = C[idx+1];
    float cz = C[idx+2];

    float A_dot_C = ax*cx + ay*cy + az*cz;
    float A_dot_B = ax*bx + ay*by + az*bz;

    ref[idx]   = bx*A_dot_C - cx*A_dot_B;
    ref[idx+1] = by*A_dot_C - cy*A_dot_B;
    ref[idx+2] = bz*A_dot_C - cz*A_dot_B;
}

std::cout << "synchronize input buffer data to device global memory\n";
boA.sync(XCL_BO_SYNC_BO_TO_DEVICE);
boB.sync(XCL_BO_SYNC_BO_TO_DEVICE);
boC.sync(XCL_BO_SYNC_BO_TO_DEVICE);

std::cout << "Execution of the kernel\n";
auto run = krnl(boA, boB, boC, boOUT, DATA_SIZE);
run.wait();

std::cout << "Get the output data from the device" << std::endl;
boOUT.sync(XCL_BO_SYNC_BO_FROM_DEVICE);


for (int i = 0; i < DATA_SIZE * 3; i++) {
    if (std::fabs(OUT[i] - ref[i]) > 1e-4) {
        std::cout << "Mismatch at " << i
```

```
                << " Expected: " << ref[i]
                << " Got: " << OUT[i] << std::endl;
            return EXIT_FAILURE;
        }
    }

    std::cout << "TEST PASSED\n";
    return 0;
}

// vadd.cpp listing
#include <stdint.h>
#include <hls_stream.h>

#define DATA_SIZE 4096

const int c_size = DATA_SIZE * 3;

static void read_input(float* in, hls::stream<float>& inStream, int size) {
mem_rd:
    for (int i = 0; i < size * 3; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        inStream << in[i];
    }
}

static void compute_triple(
    hls::stream<float>& A_stream,
    hls::stream<float>& B_stream,
    hls::stream<float>& C_stream,
    hls::stream<float>& outStream,
    int size) {

execute:
    for (int i = 0; i < size; i++) {
```

```
#pragma HLS LOOP_TRIPCOUNT min = DATA_SIZE max = DATA_SIZE
#pragma HLS PIPELINE

        // Read one vector (3 elements)
        float ax = A_stream.read();
        float ay = A_stream.read();
        float az = A_stream.read();

        float bx = B_stream.read();
        float by = B_stream.read();
        float bz = B_stream.read();

        float cx = C_stream.read();
        float cy = C_stream.read();
        float cz = C_stream.read();

        // Dot products
        float A_dot_C = ax*cx + ay*cy + az*cz;
        float A_dot_B = ax*bx + ay*by + az*bz;

        // Output result (3 values)
        outStream << (bx*A_dot_C - cx*A_dot_B);
        outStream << (by*A_dot_C - cy*A_dot_B);
        outStream << (bz*A_dot_C - cz*A_dot_B);
    }
}

static void write_result(float* out, hls::stream<float>& outStream, int size) {
mem_wr:
    for (int i = 0; i < size * 3; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        out[i] = outStream.read();
    }
}
```

```
extern "C" {

void vadd(float* in1,
        float* in2,
        float* in3,
        float* out,
        int size) {

#pragma HLS INTERFACE m_axi port=in1 bundle=gmem0
#pragma HLS INTERFACE m_axi port=in2 bundle=gmem1
#pragma HLS INTERFACE m_axi port=in3 bundle=gmem2
#pragma HLS INTERFACE m_axi port=out bundle=gmem3

#pragma HLS INTERFACE s_axilite port=in1
#pragma HLS INTERFACE s_axilite port=in2
#pragma HLS INTERFACE s_axilite port=in3
#pragma HLS INTERFACE s_axilite port=out
#pragma HLS INTERFACE s_axilite port=size
#pragma HLS INTERFACE s_axilite port=return

   static hls::stream<float> A_stream("A_stream");
   static hls::stream<float> B_stream("B_stream");
   static hls::stream<float> C_stream("C_stream");
   static hls::stream<float> outStream("out_stream");

#pragma HLS dataflow

   read_input(in1, A_stream, size);
   read_input(in2, B_stream, size);
   read_input(in3, C_stream, size);

   compute_triple(A_stream, B_stream, C_stream, outStream, size);

   write_result(out, outStream, size);
}
```

}

// make all TARGET=hw_emu

brandon — ssh bnguyen@aptd.sdsu.edu — 80×24

~ — ssh bnguyen@aptd.sdsu.edu

```
x/logs/package
Running Dispatch Server on port: 35805
INFO: [v++ 60-1548] Creating build summary session with primary output /tools/Xi
linx/home/bnguyen/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_
gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary, at Sun Apr 26 14:03:19 2026
INFO: [v++ 60-1315] Creating rulecheck session with output '/tools/Xilinx/home/b
nguyen/Vitis_Accel_Examples/hello_world/_x/reports/package/v++_package_vadd_guid
ance.html', at Sun Apr 26 14:03:19 2026
INFO: [v++ 60-895]   Target platform: /opt/xilinx/platforms/xilinx_u280_gen3x16_
xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm
INFO: [v++ 60-1578]   This platform contains Xilinx Shell Archive '/opt/xilinx/p
latforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'
INFO: [v++ 74-78] Compiler Version string: 2022.2
INFO: [v++ 60-2256] Packaging for hardware emulation
INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
 /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.
xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the re
levant reports. Run the following command.
    vitis_analyzer /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/b
uild_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary
INFO: [v++ 60-791] Total elapsed time: 0h 0m 16s
INFO: [v++ 60-1653] Closing dispatch client.
bnguyen@aptd:~/Vitis_Accel_Examples/hello_world$
```

// make run TARGET=hw_emu

brandon — ssh bnguyen@aptd.sdsu.edu — 80×24

~ — ssh bnguyen@aptd.sdsu.edu

```
x/logs/package
Running Dispatch Server on port: 35805
INFO: [v++ 60-1548] Creating build summary session with primary output /tools/Xi
linx/home/bnguyen/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_
gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary, at Sun Apr 26 14:03:19 2026
INFO: [v++ 60-1315] Creating rulecheck session with output '/tools/Xilinx/home/b
nguyen/Vitis_Accel_Examples/hello_world/_x/reports/package/v++_package_vadd_guid
ance.html', at Sun Apr 26 14:03:19 2026
INFO: [v++ 60-895]   Target platform: /opt/xilinx/platforms/xilinx_u280_gen3x16_
xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm
INFO: [v++ 60-1578]   This platform contains Xilinx Shell Archive '/opt/xilinx/p
latforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'
INFO: [v++ 74-78] Compiler Version string: 2022.2
INFO: [v++ 60-2256] Packaging for hardware emulation
INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
 /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.
xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the re
levant reports. Run the following command.
    vitis_analyzer /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/b
uild_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary
INFO: [v++ 60-791] Total elapsed time: 0h 0m 16s
INFO: [v++ 60-1653] Closing dispatch client.
bnguyen@aptd:~/Vitis_Accel_Examples/hello_world$
```

// make all TARGET=hw

brandon — ssh bnguyen@aptd.sdsu.edu — 80×24

~ — ssh bnguyen@aptd.sdsu.edu

```
x/logs/package
Running Dispatch Server on port: 33197
INFO: [v++ 60-1548] Creating build summary session with primary output /tools/Xi
linx/home/bnguyen/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3
x16_xdma_1_202211_1/vadd.xclbin.package_summary, at Sun Apr 26 15:42:33 2026
INFO: [v++ 60-1315] Creating rulecheck session with output '/tools/Xilinx/home/b
nguyen/Vitis_Accel_Examples/hello_world/_x/reports/package/v++_package_vadd_guid
ance.html', at Sun Apr 26 15:42:33 2026
INFO: [v++ 60-895]   Target platform: /opt/xilinx/platforms/xilinx_u280_gen3x16_
xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm
INFO: [v++ 60-1578]   This platform contains Xilinx Shell Archive '/opt/xilinx/p
latforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'
INFO: [v++ 74-78] Compiler Version string: 2022.2
INFO: [v++ 60-2256] Packaging for hardware
INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
 /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/./build_dir.hw.xili
nx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the re
levant reports. Run the following command.
    vitis_analyzer /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/b
uild_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary
INFO: [v++ 60-791] Total elapsed time: 0h 0m 14s
INFO: [v++ 60-1653] Closing dispatch client.
bnguyen@aptd:~/Vitis_Accel_Examples/hello_world$
```

// make run TARGET=hw

brandon — ssh bnguyen@aptd.sdsu.edu — 80×24

~ — ssh bnguyen@aptd.sdsu.edu

```
INFO: [v++ 74-78] Compiler Version string: 2022.2
INFO: [v++ 60-2256] Packaging for hardware
INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
 /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/./build_dir.hw.xili
nx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the re
levant reports. Run the following command.
    vitis_analyzer /tools/Xilinx/home/bnguyen/Vitis_Accel_Examples/hello_world/b
uild_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary
INFO: [v++ 60-791] Total elapsed time: 0h 0m 14s
INFO: [v++ 60-1653] Closing dispatch client.
bnguyen@aptd:~/Vitis_Accel_Examples/hello_world$ make run TARGET=hw PLATFORM=/op
t/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_
1_202211_1.xpfm
./hello_world_xrt -x ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xcl
bin
Open the device0
Load the xclbin ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
Allocate Buffer in Global Memory
synchronize input buffer data to device global memory
Execution of the kernel
Get the output data from the device
TEST PASSED
bnguyen@aptd:~/Vitis_Accel_Examples/hello_world$
```

2) Discussion

In P10, we are tasked with modifying the simple vector addition example to compute a vector triple product defined as the cross product of one vector with the cross product of the other two. First, we ssh into the APTD server and git clone the example repository from Github. Then we cd into our desired starting spot and run the “make clean” command to start from scratch. After that, we have 4 different commands to run in order to see how the entire workflow works. The first command is meant to *build* the vector addition example in hardware emulation mode. The second command is meant to *run* the vector addition example in hardware emulation mode. The third command is meant to synthesize the logic that implements vector addition in the FPGA. Lastly, the fourth and final command is meant to run in hardware mode where the vector addition kernel logic executes within the FPGA. Once we see TEST PASSED, we know the workflow process was conducted correctly. Most importantly, we are tasked to edit the two .cpp files in the src folder to implement the vector triple product outlined in the prompt, and then repeat the steps from the initial run. With that being said, the tenth programming assignment was useful in helping us understand the full FPGA accelerated computing flow using OpenCL/Vitis on the U280, specifically how C++ kernels can be synthesized into FPGA logic/HDL.